

JavaScript Fundamentals & the DOM

ResNet Academy | Frontend Development

Variables and types

```
const fixed = "cannot be reassigned";    let counter = 0;    // avoid var
typeof "a" // "string"    typeof 1 // "number"    typeof true // "boolean"
Falsy: false 0 "" null undefined NaN    |    Use === and !==, not == and !=
`Template ${literal}`    cond ? a : b    a ?? b    (default if null/undefined)
```

Functions

```
function add(a, b = 0) { return a + b; }
const double = (n) => n * 2;
const greet = (name) => { return `Hi ${name}`; };
array.forEach((item) => { /* callback */ });
```

Array methods

Method	Result
<code>map(fn)</code>	New array of transformed items
<code>filter(fn)</code>	New array of items passing the test
<code>find(fn)</code>	First matching item or <code>undefined</code>
<code>reduce(fn, start)</code>	One accumulated value
<code>some(fn) / every(fn)</code>	<code>true</code> if any / all items pass
<code>includes(x)</code>	<code>true</code> if the array contains <code>x</code>
<code>push(x) pop()</code>	Add / remove the last item (mutates)
<code>[...a, x]</code>	Copy with an added item (no mutation)

Objects and JSON

```
const { title, level } = course;    // destructuring
const copy = { ...course, level: "x" };    // spread and override
Object.keys(course)    Object.values(course)    Object.entries(course)
JSON.stringify(obj)    // object -> text    JSON.parse(text)    // text -> object
```

DOM selection and changes

Task	Code
Select one / many	<code>document.querySelector(css) / querySelectorAll(css)</code>
Set text safely	<code>el.textContent = value</code>
Classes	<code>el.classList.add / remove / toggle("name")</code>
Attributes / data	<code>el.setAttribute("aria-live", "polite"), el.dataset.id</code>
Create and add	<code>const li = document.createElement("li"); ul.append(li)</code>
Remove / clear	<code>el.remove(), ul.replaceChildren()</code>

Key idea. Never assign untrusted input to `innerHTML`. Use `textContent`. `querySelector` returns `null` when nothing matches, so check first.

Events

```
el.addEventListener("click", (event) => { /* ... */ });
form.addEventListener("submit", (event) => { event.preventDefault(); });
list.addEventListener("click", (e) => { const li = e.target.closest("li"); });    // delegation
```

Event	When
<code>click</code>	Click, tap or keyboard activation
<code>input</code>	Value changes (every keystroke)
<code>change</code>	Value committed

Event	When
submit	Form submitted (listen on the form)
keydown	Key pressed

Form validation

```
input.checkValidity()           // true if all constraints pass
input.validity.valueMissing     // required and empty
input.validity.typeMismatch     // for example a bad email
input.validity.tooShort         // below minlength
input.setCustomValidity("msg") // custom error, "" clears it
Object.fromEntries(new FormData(form)) // form values as an object
```

Key idea. Client-side validation is for user experience only. The server must always validate again.